

Introduction to Decision-Focused Learning for airlines

Part 1: delay prediction, stochastic planning, and column generation

Léo Baty

22/09/2026



ÉCOLE NATIONALE DES
PONTS
ET CHAUSSÉES



IP PARIS

AIRFRANCE 

Agenda for the day

Part 1: OR

1. The practical problem
2. Classical OR: column generation
3. Practice session 1: OR for tail assignment in practice

Part 2: DFL

4. Intro to Decision-Focused Learning
5. Demo on a toy problem
6. Practice session 2: DFL for tail assignment in practice

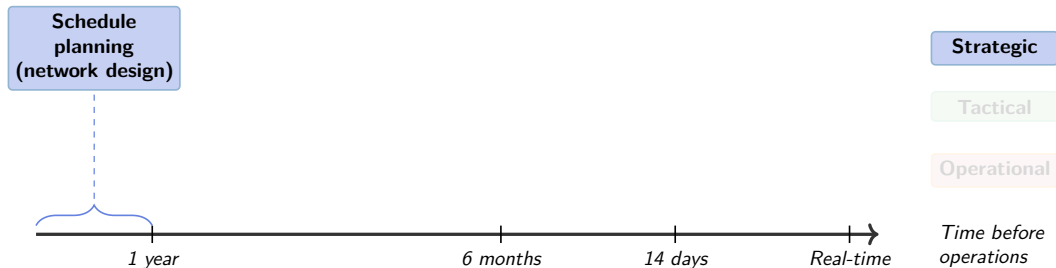
Part 1: outline

- 1 The stochastic tail assignment problem
- 2 Modeling delays
- 3 Optimizing with column generation
- 4 Practice session 1

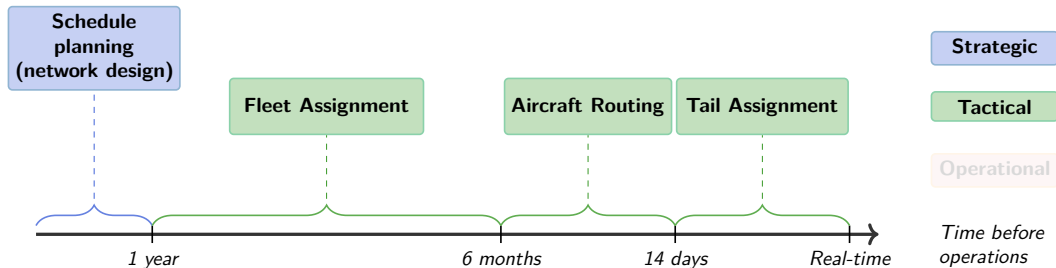
Overview

- 1 The stochastic tail assignment problem
- 2 Modeling delays
- 3 Optimizing with column generation
- 4 Practice session 1

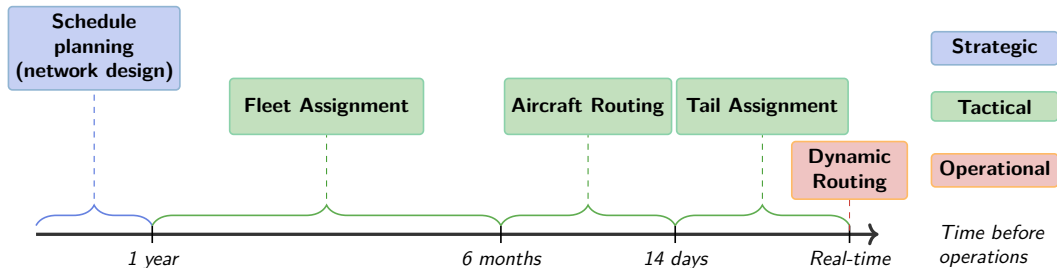
Airline operations at Air France



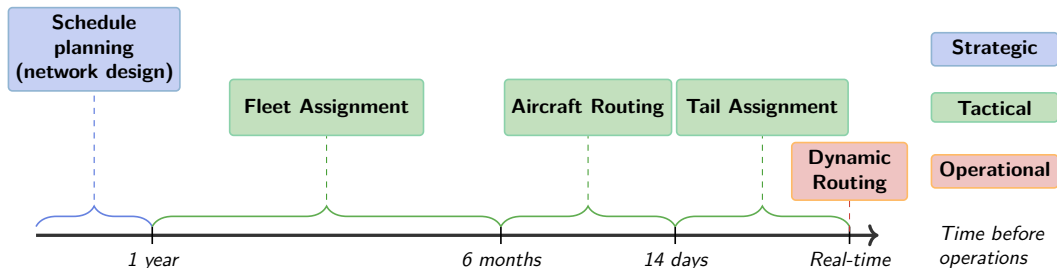
Airline operations at Air France



Airline operations at Air France

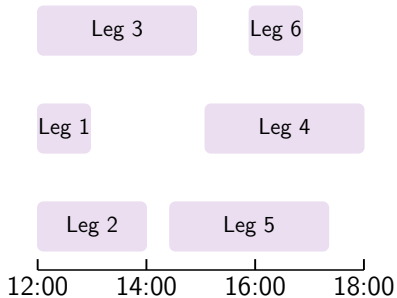


Airline operations at Air France



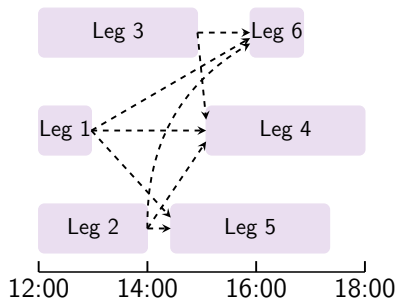
- Focus on **tail assignment**
- Generic methodology, can be applied to other problems

The (deterministic) tail assignment problem



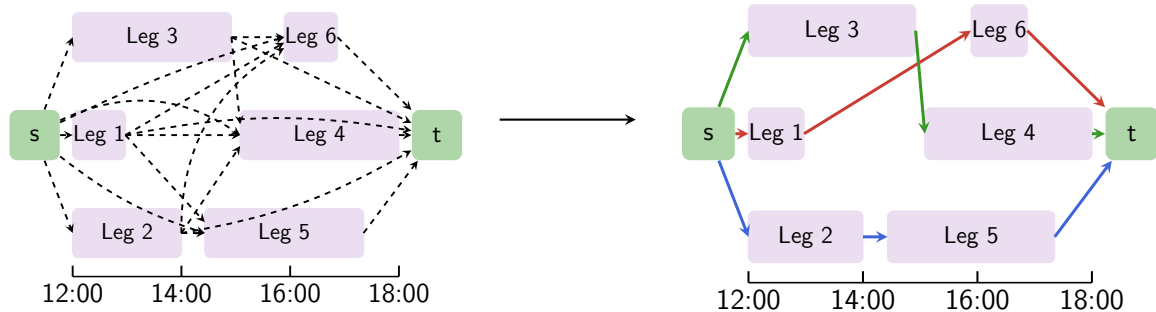
Flight Leg	Departure	Arrival
1: B→A	12:00	13:00
2: C→A	12:00	14:00
3: D→A	12:00	15:00
4: A→D	15:10	18:10
5: A→C	14:30	17:30
6: A→B	16:00	17:00

The (deterministic) tail assignment problem

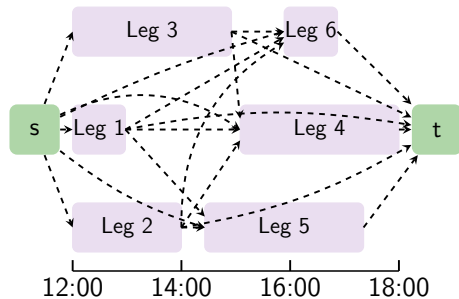


Flight Leg	Departure	Arrival
1: B→A	12:00	13:00
2: C→A	12:00	14:00
3: D→A	12:00	15:00
4: A→D	15:10	18:10
5: A→C	14:30	17:30
6: A→B	16:00	17:00

The (deterministic) tail assignment problem



The (deterministic) tail assignment problem

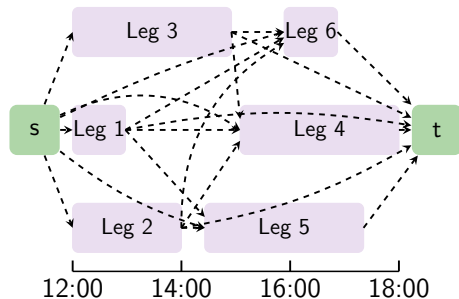


Aircraft	Fuel Factor
1	1.0
2	2.0
3	3.0

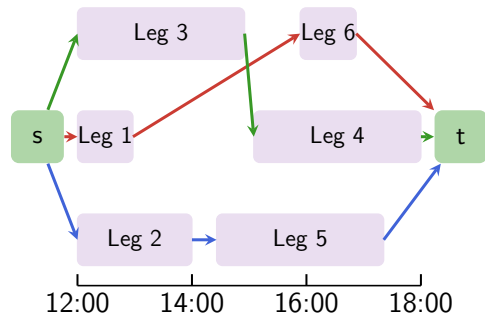
$\min_{\text{routes } y}$ $\overbrace{\text{fuel cost} + \text{connection cost}}^{\text{operational cost}}$

subject to $\left\{ \begin{array}{l} \text{valid routes,} \\ \text{maintenance constraints,} \\ \text{mandatory connections.} \end{array} \right.$

The (deterministic) tail assignment problem



$$\min_{y \in \mathcal{Y}(x)} \theta^\top y$$

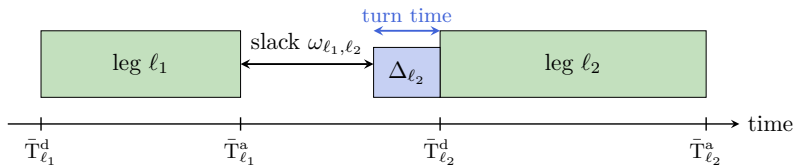


Aircraft	Fuel Factor
1	1.0
2	2.0
3	3.0

- **Deterministic** problem: easily solved by commercial MILP solvers

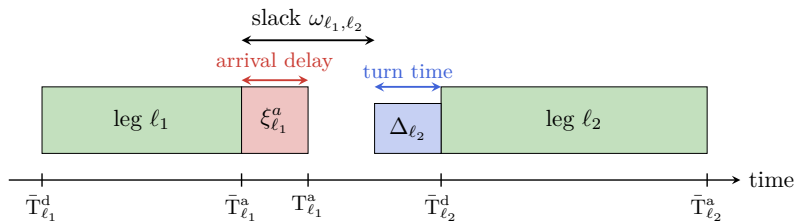
Delay propagation

- A connection along an aircraft route:



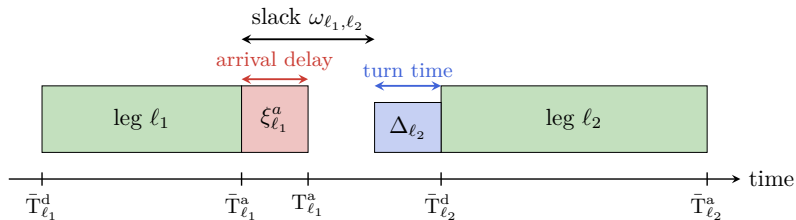
Delay propagation

- No delay propagation:

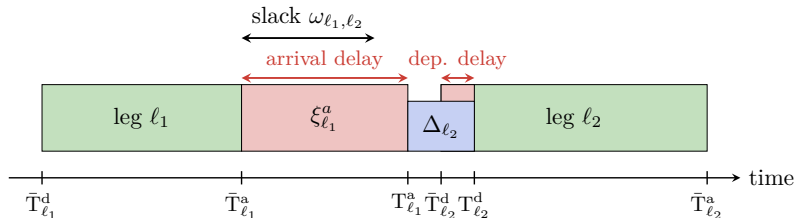


Delay propagation

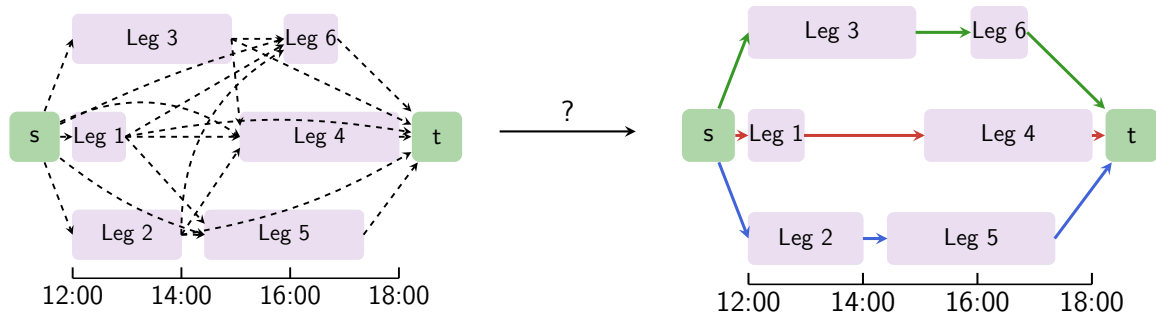
■ No delay propagation:



■ Delay propagation:



The **stochastic** tail assignment problem



- **Deterministic** problem: easily solved by commercial MILP solvers
- Does not take **delays** into account, minimum turn time is not enough
- **Stochastic** problem: non-linear

The **stochastic** tail assignment problem

$\min_{\text{routes } y}$ operational cost + $\mathbb{E}_{\text{delays}}[\text{delays cost}]$

subject to $\left\{ \begin{array}{l} \text{valid routes,} \\ \text{maintenance constraints,} \\ \text{mandatory connections.} \\ \text{etc.} \end{array} \right.$

■ Delays cost:

- ▶ customer dissatisfaction and compensation
- ▶ additional operational costs

Can we leverage historical data and recent advances in machine learning to become data-driven and scale up?

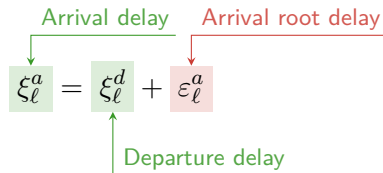
How to scale on large-scale industrial instances for stochastic problems ?



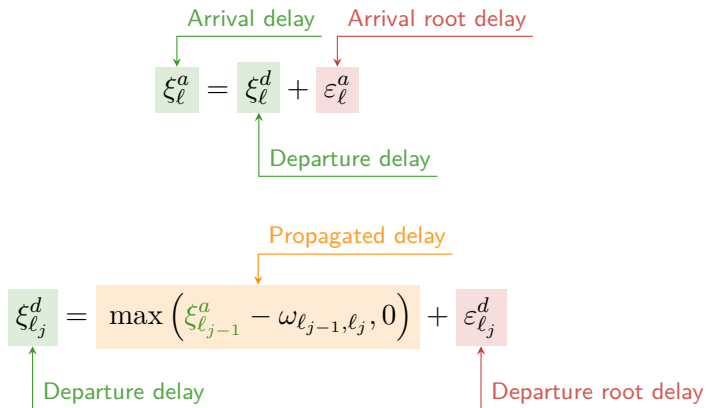
Overview

- 1 The stochastic tail assignment problem
- 2 Modeling delays
- 3 Optimizing with column generation
- 4 Practice session 1

Delay propagation along aircraft routes



Delay propagation along aircraft routes



Modeling intrinsic delays

- **Root delays** modeled as (log-)normal distributions:

$$\varepsilon_{\ell}^a \sim \mathcal{N}(\mu_{\ell}^a, \sigma_{\ell}^a),$$

$$\varepsilon_{\ell}^d \sim \log \mathcal{N}(\mu_{\ell}^d, \sigma_{\ell}^d)$$

Any other **differentiable** distribution can be used instead.

Modeling intrinsic delays

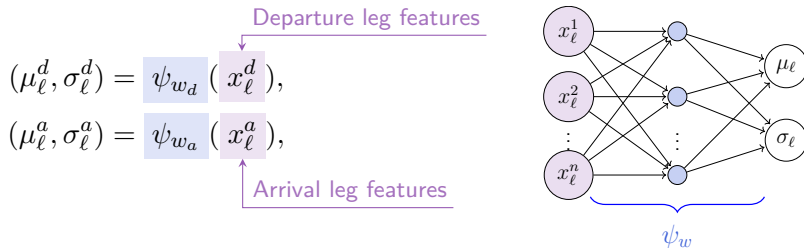
- **Root delays** modeled as (log-)normal distributions:

$$\varepsilon_{\ell}^a \sim \mathcal{N}(\mu_{\ell}^a, \sigma_{\ell}^a),$$

$$\varepsilon_{\ell}^d \sim \log \mathcal{N}(\mu_{\ell}^d, \sigma_{\ell}^d)$$

Any other **differentiable** distribution can be used instead.

- Distribution parameters are parameterized by **neural networks**:



Modeling intrinsic delays

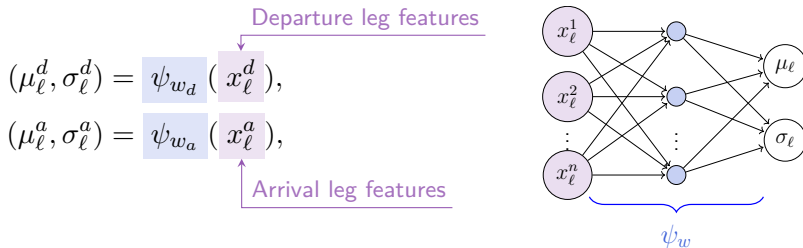
- **Root delays** modeled as (log-)normal distributions:

$$\varepsilon_{\ell}^a \sim \mathcal{N}(\mu_{\ell}^a, \sigma_{\ell}^a),$$

$$\varepsilon_{\ell}^d \sim \log \mathcal{N}(\mu_{\ell}^d, \sigma_{\ell}^d)$$

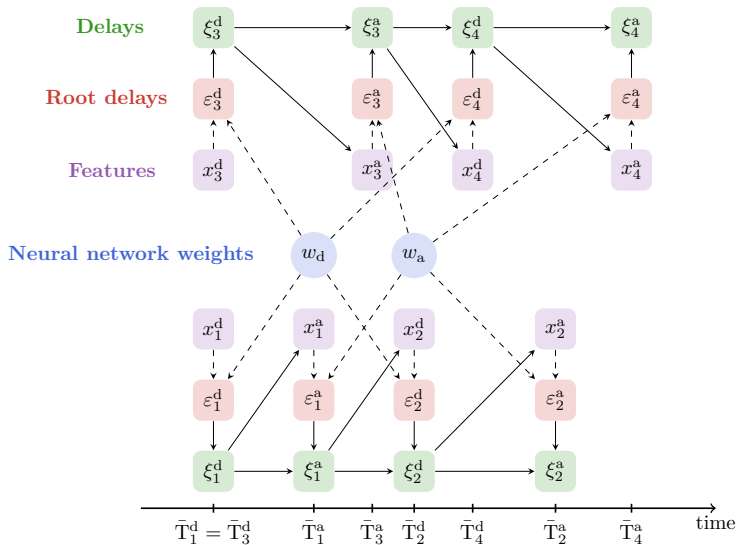
Any other **differentiable** distribution can be used instead.

- Distribution parameters are parameterized by **neural networks**:



- **Goal:** how to learn parameters w_d and w_a ?

Delay prediction model



Log-likelihood maximization

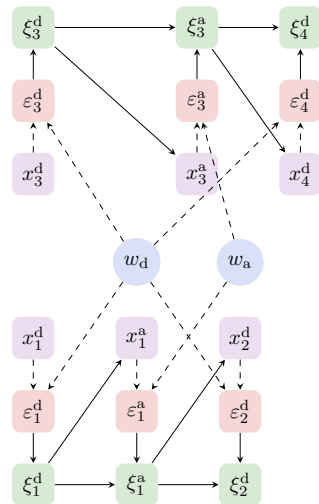
When the Bayesian Network is an **acyclic graph**:

$$p(v_1, \dots, v_n) = \prod_{i=1}^n p(v_i \mid \text{parents}(v_i))$$

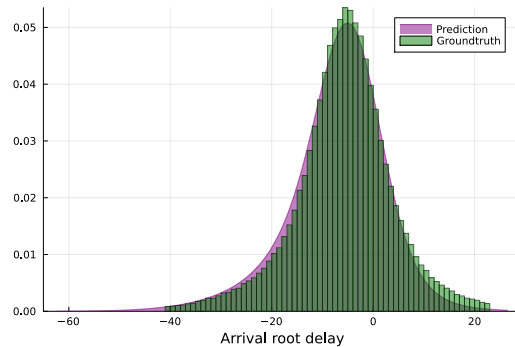
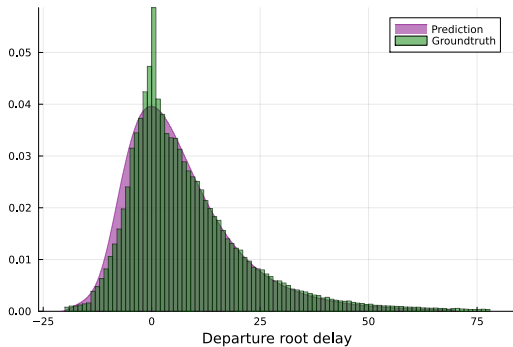
Log-likelihood of neural network parameters:

$$l(w; x, \varepsilon) = \sum_{\ell \in \mathcal{L}} \log p(\varepsilon_{\ell}^d | x_{\ell}^d, w) + \log p(\varepsilon_{\ell}^a | x_{\ell}^a, w)$$

$\Rightarrow$ **gradient descent** optimization can be used



Results



Overview

- 1 The stochastic tail assignment problem
- 2 Modeling delays
- 3 Optimizing with column generation
- 4 Practice session 1

Compact MIP

c^0 is **non-linear** in the leg-level variables y_a^i :

- delay propagation along a route is non-linear (recursive)
- delay cost is piecewise linear

$\Rightarrow$ an exact compact MILP exists (per-scenario delay variables, linearized propagation), but needs $\sim$ **arcs** $\times$ **scenarios** extra variables and has a **weak LP relaxation**: does not scale past a few hundred legs, even with 1 scenario.

$$\begin{aligned}
 & \min_y \quad \frac{1}{S} \sum_{s=1}^S c^0(y; x, \xi_s) \\
 & \text{s.t.} \quad \sum_{a \in \delta^-(v) \cap \mathcal{A}^i} y_a^i = \sum_{a \in \delta^+(v) \cap \mathcal{A}^i} y_a^i, \forall v \in \mathcal{V}, \forall i \in \mathcal{I}, \\
 & \quad \sum_{a \in \delta^+(o) \cap \mathcal{A}^i} y_a^i = \sum_{a \in \delta^-(t) \cap \mathcal{A}^i} y_a^i = 1, \forall i \in \mathcal{I}, \\
 & \quad \sum_{i \in \mathcal{I}} \sum_{a \in \delta^-(\ell) \cap \mathcal{A}^i} y_a^i = 1, \quad \forall \ell \in \mathcal{L}, \\
 & \quad y_a^i \in \{0, 1\}, \forall i \in \mathcal{I}, \forall a \in \mathcal{A}^i.
 \end{aligned}$$

Total cost, non-linear (delay propagation)

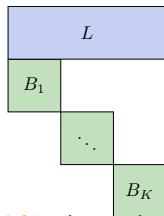
Flow constraints

Set-partitioning constraints

Route reformulation

Dantzig-Wolfe decomposition: one variable per feasible **route** $r \in \mathcal{R}^i$ instead of one per leg.

- route cost c_r^i hides the non-linearity:
computed by **simulation** over scenarios,
once the route is fixed
- deals with delay propagation exactly



Master problem:

$$\min_y \sum_{i \in \mathcal{I}} \sum_{r \in \mathcal{R}^i} c_r^i y_r^i$$

Each leg covered exactly once

$$\text{s.t.} \quad \sum_{i \in \mathcal{I}} \sum_{r \ni \ell, r \in \mathcal{R}^i} y_r^i = 1, \quad \forall \ell \in \mathcal{L},$$

$$\sum_{r \in \mathcal{R}^i} y_r^i = 1, \quad \forall i \in \mathcal{I},$$

$$y_r^i \in \{0, 1\}, \quad \forall i \in \mathcal{I}, \forall r \in \mathcal{R}^i.$$

$|\mathcal{R}^i|$ is **exponential** in the number of legs $\Rightarrow$ generate routes on the fly instead of enumerating them. Column generation solves the **LP relaxation** of the master (replace $y_r^i \in \{0, 1\}$ by $y_r^i \geq 0$). Integrality is recovered afterwards.

Column generation: the algorithm

Only a handful of the exponentially many routes are used at the LP optimum $\implies$ work with a **restricted master problem** over a small set of generated routes $\tilde{\mathcal{R}}^i \subseteq \mathcal{R}^i$, and add routes only when they can improve the LP.

```
initialize  $\tilde{\mathcal{R}}^i$  with a few feasible routes for each  $i \in \mathcal{I}$ ;  
repeat  
  solve the restricted master LP over the current sets  
     $(\tilde{\mathcal{R}}^i)_{i \in \mathcal{I}}$ ;  
  get duals  $\lambda_\ell$  ( $\ell \in \mathcal{L}$ ) and  $\mu_i$  ( $i \in \mathcal{I}$ );  
  for each aircraft  $i \in \mathcal{I}$  do  
    pricing:  $r^* \in \operatorname{argmin}_{r \in \mathcal{R}^i} c_r^i - \sum_{\ell \in r} \lambda_\ell - \mu_i$ ;  
    if the minimum is  $< 0$  then  
      | add  $r^*$  to  $\tilde{\mathcal{R}}^i$ ;  
    end  
  end  
until no route with negative reduced cost is found;  
return LP optimum (lower bound) and fractional  $y$ 
```

- **Restricted master**: same LP as before, but on generated routes only, so it stays small
- **Duals**: λ_ℓ is the price of covering leg ℓ , μ_i the price of using aircraft i
- **Pricing**: finds the best route for the current prices *without* enumerating $\mathcal{R}^i$: this is where the structure of the problem lives (next slide)
- **Stopping rule**: by LP duality, no negative reduced cost means the restricted LP optimum is optimal for the full master

Pricing: a stochastic shortest path

Duals of the master LP:

- λ_ℓ : price of covering leg ℓ
- μ_i : price of assigning a route to aircraft i

Reduced cost of route $r \in \mathcal{R}^i$:

$$\bar{c}_r^i = c_r^i - \sum_{\ell \in r} \lambda_\ell - \mu_i.$$

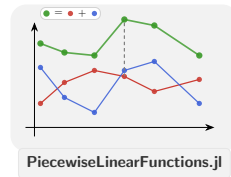
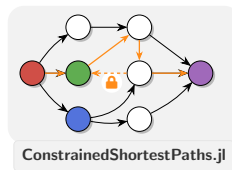
Pricing sub-problem for aircraft $i \in \mathcal{I}$:

$\min_{r \in \mathcal{R}^i} \bar{c}_r^i$. A column is added when $\bar{c}_r^i < 0$.

$\implies$ a shortest path with a **non-additive resource** (the propagated delay in each scenario), solved by **label-setting** with **dominance** and **bounding**.

Enumeration algorithm:

- with **dominance**
- and **bounding**, with convex piecewise linear functions as resources



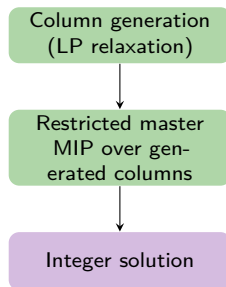
From fractional to integer: restricted master heuristic

Column generation returns a fractional LP solution y (with the **LP relaxation lower bound**): we need an integer one.

Restricted master heuristic: keep the columns generated so far, restore $y_r^i \in \{0, 1\}$, and solve as a **MIP** with an off-the-shelf solver.

Pros: simple, cheap in principle (columns are already generated), any feasible solution gives an **upper bound**.

Cons: the columns were selected to solve the LP, not to combine into an integer solution $\implies$ a good integer solution may simply not be in the pool. In our results this gives gaps of **74% to 147%**, and the MIP often hits the **one-hour time limit** on large instances.



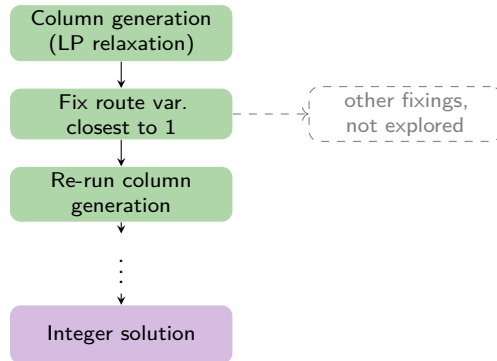
From fractional to integer: diving heuristic

Diving: **depth-first** descent in the branch-and-price tree, with **limited backtracking**.

At each node:

1. solve the LP by column generation
2. fix the route variable y_r^i with the largest fractional value to 1, removing aircraft i and its legs from the graph
3. re-run column generation on the reduced problem, reusing the still-compatible columns
4. if a node is infeasible, apply **limited backtracking**: try the next-largest fractional route instead, up to a taboo list of size k ($k = 15$ in the paper)
5. repeat until the solution is integer (at most $|\mathcal{I}|$ fixings)

Shrinking the graph makes column generation **cheaper** at deeper nodes: on average diving costs about one to two column generation runs, and reaches gaps of **a few tenths of a percent** instead of 74% to 147%, at the price of much longer worst cases on the largest instances.



Branch-and-price: the exact alternative

- Branch-and-bound where **column generation is solved at every node**
- Branching rule must stay compatible with the pricing problem: branch on **arcs of the flight graph**, not on route variables y_r^i
 - ▶ fixing $y_r^i = 1$ (as diving does) is compatible with pricing: it removes aircraft i and its legs from the graph
 - ▶ a branch $y_r^i = 0$ would require excluding one specific path from the pricing, which label-setting cannot express, hence an exact tree branches on arcs instead
- Gives an **optimality proof** (exact method), but cost grows with the size of the search tree

	Exact?	Computational cost	Solution quality
Restricted master heuristic	✗	low (but hits the time limit on large instances)	poor (large gap)
Diving	✗	moderate	near-optimal
Branch-and-price	✓	high (tree size)	optimal

Diving is the practical choice at Air France scale: it already closes the gap to the column generation lower bound to a few tenths of a percent, while each branch-and-price node costs a full stochastic column generation run (minutes to hours) $\implies$ the tree is unaffordable.

Results

Air France instances, small to large (60 to 600 flight legs), 129 instances in total. Gaps are relative to the **column generation lower bound**. Restricted master heuristic times are capped by a **one-hour time limit**.

- Column generation gives a **lower bound**

Results

Air France instances, small to large (60 to 600 flight legs), 129 instances in total. Gaps are relative to the **column generation lower bound**. Restricted master heuristic times are capped by a **one-hour time limit**.

- Column generation gives a **lower bound**
- Restricted master heuristic

Scenarios	Column generation	Restricted master heuristic	
	Average time	Average time	Average gap
1	578s	379s	74%
5	667s	654s	137%
10	915s	780s	147%
50	1596s	638s	143%
100	2738s	568s	117%

Results

Air France instances, small to large (60 to 600 flight legs), 129 instances in total. Gaps are relative to the **column generation lower bound**. Restricted master heuristic times are capped by a **one-hour time limit**.

- Column generation gives a **lower bound**
- Restricted master heuristic
- Diving heuristic

Scenarios	Column generation	Diving heuristic	
	Average time	Average time	Average gap
1	578s	870s	0.1%
5	667s	1381s	0.2%
10	915s	818s	0.4%
50	1596s	1338s	0.2%
100	2738s	2061s	0.3%

What we have now

- **Near-optimal solutions** (a few tenths of a percent from the column generation lower bound), but **hours per instance**, growing with the number of legs and scenarios
- A **lower bound** from the column generation (LP) relaxation
- The ability to build a **dataset** of (instance, solution) pairs
- $\implies$ used this afternoon to **learn** a faster decision-making pipeline

Practice session 1

See 02_or_notebook.jl